Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Науково-навчальний інститут фізико-технічний інститут
Кафедра інформаційної безпеки

Лабораторна робота №1

з дисципліни «Технологія блокчейн та розподілені системи»

на тему:

«Дослідження систем Etherium та інших криптовалют»

Виконали:
студенти групи ФБ-51мп
Спускан Дарина Михайлівна,
Свердлик Іван Васильович

Перевірив:
асистент кафедри ММЗІ
Селюх Поліна Валентинівна

Дата захисту _____

Захищено з балом _____

Київ 2026

Мета: дослідити технічні особливості розгортання блокчейн-платформ, провести порівняльний аналіз транзакційних моделей і обґрунтувати архітектурну несумісність їхніх програмних модулів.

Завдання:

Провести порівняльний аналіз особливостей розгортання систем криптовалют у порівнянні із системою Etherium. Зробити висновок про можливість чи неможливість взаємозаміни модулів різних систем та пояснити причини.

Хід роботи

Аналіз архітектурної моделі даних

Архітектурна модель даних визначає фундаментальний спосіб збереження, верифікації та передачі інформації про стан активів у розподіленому реєстрі. Вибір моделі безпосередньо впливає на масштабованість системи, складність розгортання вузлів та логіку роботи смартконтрактів. У сучасному блокчейні домінують два підходи: UTXO та Account-based.

Модель UTXO

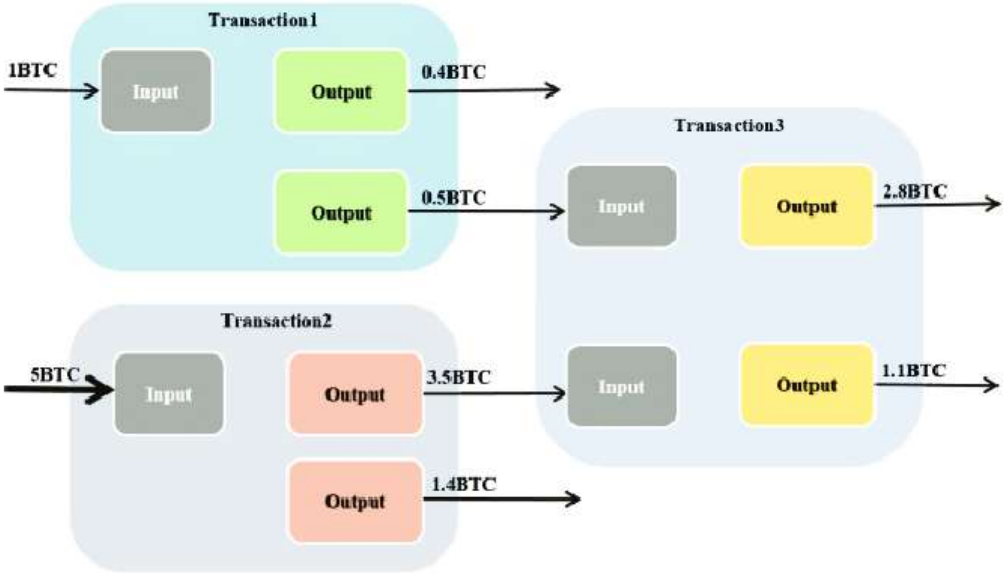

Рисунок 1. Модель UTXO

Модель UTXO базується на концепції невитрачених виходів транзакцій. У цій парадигмі в блокчейні не існує такого поняття, як баланс гаманця у вигляді єдиного числа. Замість цього стан мережі – це величезна сукупність окремих цифрових купюр, виходів, кожна з яких має свого власника. Системи, що побудовані на моделі UTXO: Bitcoin, Litecoin, Dash.

Кожна транзакція створює один або декілька виходів. Кожен вихід містить суму, кількість монет, та криптографічний замок, скрипт, який визначає умови витрачання цих коштів, зазвичай наявність цифрового підпису власника. Щоб здійснити нову транзакцію, користувач повинен надати вхід, який є посиланням на існуючий невитрачений вихід попередньої транзакції. Коли ж вихід використовується як вхід, він стає «витраченим» і назавжди вилучається з активного набору UTXO. Якщо сума виходу більша за необхідну для оплати, система автоматично створює новий вихід – решту, яка повертається на адресу відправника.

Переваги та особливості розгортання такої моделі:

- користувач може використовувати нову адресу для кожного отриманого виходу, що ускладнює відстеження потоків капіталу;
- оскільки транзакції оперують незалежними виходами, вузол може перевіряти декілька транзакцій одночасно, якщо вони не використовують одні й ті самі виходи;
- при розгортанні вузла, наприклад, Bitcoin Core, критично важливим є об'єм оперативної пам'яті для кешування набору UTXO, оскільки саме до нього система звертається найчастіше для валідації нових блоків.

Модель Account-based

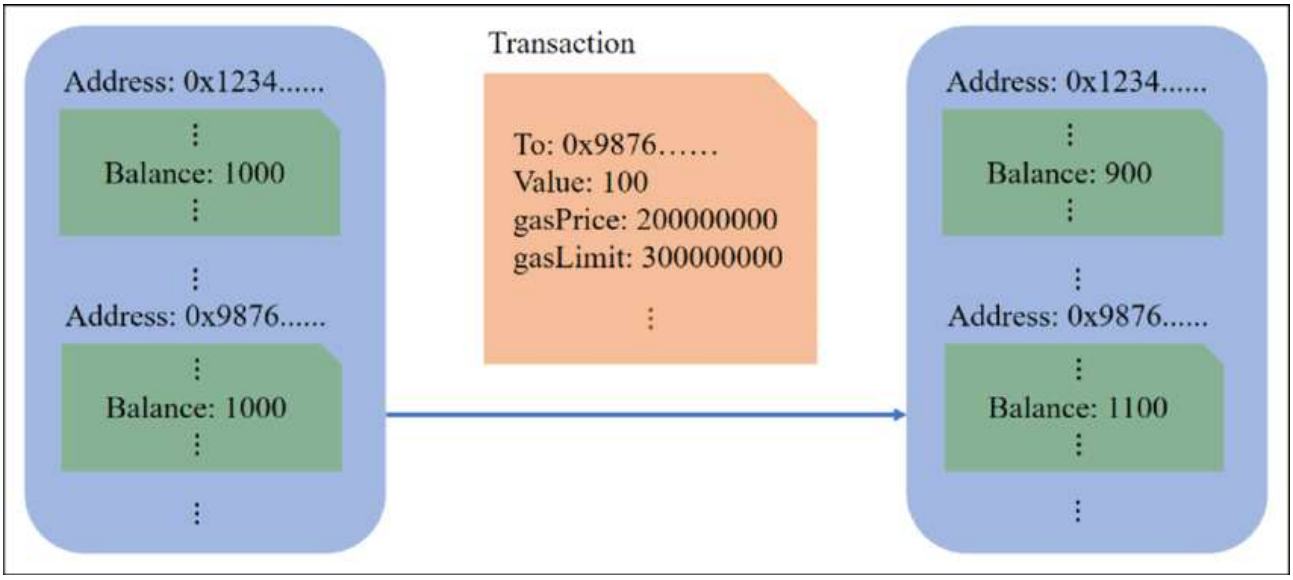

Рисунок 2. Модель account-based

Модель на основі рахунків функціонує аналогічно до традиційної банківської системи. Стан блокчейну зберігається у вигляді бази даних адрес, де кожній адресі відповідає поточний баланс, лічильник транзакцій, nonce, та, у випадку смарт-контрактів, код і сховище даних. Системи, що побудовані на моделі account-based: Ethereum, NEO.

Транзакція в такій моделі – це пряма інструкція: «Зменшити баланс рахунку А на Х та збільшити баланс рахунку Б на Х». Ця модель ідеально підходить для платформ як Ethereum та NEO, оскільки дозволяє контрактам мати власний стійкий стан. Наприклад, смарт-контракт може зберігати список учасників голосування або умови виконання угоди, які оновлюються з кожною транзакцією. На відміну від UTXO, де транзакція може мати 10 входів і 10 виходів, тут транзакція зазвичай ініціюється одним обліковим записом, що спрощує логіку написання коду.

Транзакції в моделі рахунків зазвичай менші за розміром, оскільки вони не несуть у собі посилання на численні попередні виходи. Також для захисту від повторного надсилання однієї й тієї ж транзакції використовується параметр nonce – порядковий номер транзакції для конкретного рахунку.

Структури баз даних

Тип моделі даних безпосередньо визначає, як інформація фізично записується на диск під час розгортання вузла. Більшість блокчейн-клієнтів використовують NoSQL бази даних типу ключ-значення для зберігання стану мережі.

LevelDB (Ethereum, Bitcoin) – оптимізована для швидкого запису та послідовного зчитування. Вона ефективно стискає дані, що важливо для зберігання величезної кількості дрібних станів акаунтів або виходів транзакцій.

RocksDB (NEO, деякі клієнти Ethereum) – це форк LevelDB від Meta, оптимізований для роботи на багатоядерних процесорах та швидких SSD-накопичувачах. Вона дозволяє швидше проводити початкову синхронізацію, що критично для систем з високою пропускною здатністю як NEO.

В моделі account-based база даних постійно зростає, оскільки кожен створений акаунт або контракт назавжди залишається в Global State, якщо не передбачено механізмів видалення. Це вимагає від розгортання вузла наявності швидких SSD, інакше вузол не встигатиме синхронізувати нові блоки.

В моделі UTXO дисковий простір використовується переважно для зберігання історії транзакцій. Оскільки старі транзакції рідко потрібні для поточної валідації, вимоги до швидкості доступу до старих даних нижчі, ніж у Ethereum.

При розгортанні обох систем можливе використання режиму pruning. В UTXO-системах це дозволяє видалити витрачені виходи, значно економлячи місце. В account-based системах видалення старих станів є значно складнішим технічним завданням, що робить повні вузли Ethereum більш ресурсомісткими.

Аналіз архітектурних моделей демонструє, що взаємозамінність модулів обробки даних між цими системами є неможливою. Програмний код, розрахований на перевірку цілісності ланцюга UTXO, де валідується джерело кожної цифрової монети, не здатний обробити транзакцію в моделі рахунків, де перевіряється лише кінцевий баланс та nonce. Це створює фундаментальний

розрив між логікою збереження даних у Bitcoin-подібних системах та платформах смарт-контрактів.

Механізми консенсусу та мережевої ієрархії

Успішне розгортання блокчейн-системи залежить від того, як організована її мережева ієрархія. Якщо в класичних базах даних існує принцип Master-Slave, то в децентралізованих системах ієрархія визначається алгоритмом консенсусу, який диктує права та обов'язки кожного вузла.

Однорангові мережі та Proof-of-Work (Bitcoin, Litecoin)

У класичних мережах першого покоління реалізована концепція абсолютної децентралізації, де всі повноправні вузли мають рівні права. Механізм консенсусу Proof-of-Work базується на розв'язанні інтенсивних обчислювальних задач. Майнери змагаються за право знайти хеш заголовка блоку, який буде меншим за встановлену ціль.

Типи вузлів:

- Full Node, що зберігає повну копію блокчейну, перевіряє кожну транзакцію та блок на відповідність правилам протоколу;
- Mining Node – спеціалізований вузол, який виконує обчислення, хешування.
- SPV – легкі вузли, що завантажують лише заголовки блоків. Використовуються в мобільних гаманцях.

Розгортання таких систем вимагає значних апаратних ресурсів для майнінгу, ASIC для Bitcoin, GPU/ASIC для Litecoin Scrypt. На рівні мережі це вимагає відкриття порту 8333 (Bitcoin) для забезпечення P2P-зв'язності.

Дворангові мережі

Dash впроваджує складнішу, дворівневу ієрархію мережі, що суттєво змінює підхід до розгортання інфраструктури. Перший рівень – це звичайні майнери, PoW, які забезпечують базову емісію та безпеку ланцюга. Другий рівень, Masternodes, – це спеціальні вузли, які виконують сервісні функції: миттєві транзакції та змішування монет для анонімності.

Для розгортання мастерноди необхідно мати заставу в 1000 DASH на гаманці. Це робить вузол акціонером мережі. Мастерноди голосують за розвиток проекту, що робить Dash першою успішною децентралізованою автономною організацією.

На відміну від Bitcoin, мастернода повинна мати статичну IP-адресу та працювати 24/7 з високим показником uptime, інакше вона виключається зі списку виплат.

Делегований консенсус

NEO використовує алгоритм Delegated Byzantine Fault Tolerance, який радикально відрізняється від PoW відсутністю майнінгу та високою швидкістю фіналізації.

Консенсусні вузли, Bookkeepers – це обмежена кількість вузлів, які обираються власниками токенів NEO. Саме вони формують блоки. Звичайні ж вузли виконують роль спостерігачів та точок доступу для користувачів.

Процес консенсусу виглядає наступним чином: один з консенсусних вузлів призначається спікером і пропонує новий блок; якщо 2/3 інших вузлів підтверджують його, блок вважається незворотним.

Перевагою розгортання є відсутність імовірнісного розгалуження. Як тільки блок записано, він є фінальним, а це ідеально для корпоративних систем, але вимагає високої довіри до обраних делегатів.

Proof-of-Stake та Beacon Chain

Сучасний Ethereum перейшов на архітектуру, де безпека базується не на обчисленнях, а на економічній заставі - стекінгу.

Beacon Chain – це центральний «координатор» мережі, який керує реєстром валідаторів та обирає тих, хто пропонуватиме наступний блок. Замість майнерів тепер працюють валідатори, які внесли 32 ETH у смарт-контракт. Вони отримують винагороду за чесну перевірку блоків та штрафуються за спробу маніпуляцій.

Мережева структура:

- Execution Client обробляє транзакції та смарт-контракти.
- Consensus Client керує логікою PoS та зв'язком з Beacon Chain.

Розгортання вузла Ethereum тепер вимагає синхронної роботи двох окремих програмних клієнтів, Execution + Consensus, що збільшує складність адміністрування порівняно з часами PoW.

Аналіз мережевих ієрархій підтверджує неможливість взаємозаміни модулів через несумісність логіки прийняття рішень.

- У Bitcoin вузол ідентифікується за IP, у Dash – за транзакцією застави, в Ethereum – за публічним ключем валідатора в Beacon Chain.
- Модуль консенсусу NEO розрахований на миттєву фіналізацію, тоді як Bitcoin-подібні системи працюють за принципом імовірнісної фіналізації, потрібно чекати 6 підтверджень. Об'єднання цих модулів призведе до розсинхронізації всієї мережі.
- Також вузол Dash не зможе валідувати блок Ethereum, оскільки він не має доступу до реєстру валідаторів Beacon Chain і не розуміє механізму slashing.

Аналіз обчислювальних можливостей

Логічний шар блокчейну визначає правила, за якими активи змінюють власників, та умови, за яких виконуються транзакції. Відмінності між системами в цьому аспекті є найбільш радикальними: від простих умов підпису до складних автономних програм.

Обмежений скриптинг та мова Script (Bitcoin, Litecoin, Dash)

Системи першого покоління використовують низькорівневу мову сценаріїв, яка називається Script, стекову мову, подібну до Forth, яка виконується лінійно, зверху вниз.

У Script навмисно відсутні цикли, наприклад, for або while, та складні рекурсії. Це відповідно зроблено для безпеки: неможливо створити транзакцію, яка б зациклила мережу і зупинила роботу всіх вузлів.

Script підтримує лише базові криптографічні та логічні операції:

- OP_CHECKSIG - перевірка цифрового підпису.
- OP_CHECKMULTISIG - вимога підписів від декількох осіб.
- OP_CHECKLOCKTIMEVERIFY - блокування коштів до певної дати.

Хоча Dash базується на коді Bitcoin, його логічний шар доповнений функціями InstantSend та PrivateSend, які реалізовані на рівні протоколу через мастерноди, а не через користувацькі смарт-контракти.

Віртуальні машини та смарт-контракти

На відміну від Script, платформи Ethereum та NEO пропонують повноцінне середовище виконання – віртуальну машину. Це дозволяє розгортати смарт-контракти – автономні програми зі своїм внутрішнім станом.

EVM – це серце Ethereum. Вона є Тюрінг-повною, що дозволяє реалізувати алгоритм будь-якої складності. Оскільки EVM дозволяє цикли, існує ризик

нескінченного виконання коду. Для вирішення цієї проблеми введено газ. Кожна обчислювальна операція коштує певну кількість одиниць газу. Якщо у транзакції закінчується газ, виконання переривається, а стан повертається до початкового. EVM повністю ізольована від основної файлової системи вузла, що гарантує безпеку під час виконання коду з невідомих джерел.

NeoVM подібна до EVM за своєю суттю, але має інший підхід до взаємодії з даними. Логічний шар NEO спроектований так, щоб легше взаємодіяти з даними поза блокчейном. NeoVM використовує більш легкий набір інструкцій, що дозволяє досягати вищої пропускної здатності транзакцій порівняно з першими версіями EVM.

Підтримка мов програмування

Вибір мови програмування для розробки логіки блокчейну визначає поріг входу для розробників та безпеку коду. В Ethereum використовуються спеціалізовані мови такі як Solidity та Vyper.

Solidity – об'єктно-орієнтована мова, синтаксис якої нагадує JavaScript/C++. Вона розроблена спеціально для роботи з блокчейном, має вбудовані типи даних для адрес, балансів та криптографічних функцій.

В NEO використовуються загальні мови C#, Java, Python, Go, Rust. Це є величезною перевагою для розгортання систем, оскільки розробникам не потрібно вчити нову мову. NEO використовує компілятори, які перетворюють стандартний код у байт-код NeoVM.

В Bitcoin, Litecoin використовуються декларативні крипти. Скрипти пишуться у вигляді послідовності команд. Це вимагає глибокого розуміння протоколу, але забезпечує максимальну безпеку через простоту логіки.

Аналіз логічного шару підводить нас до фінального пояснення неможливості взаємозаміни модулів:

- модуль, що відповідає за виконання Script у Bitcoin, не має стека та логіки для обробки Gas, тому він фізично не зможе виконати навіть найпростіший смарт-контракт Ethereum;
- байт-код EVM та NeoVM має різну структуру інструкцій. Смарт-контракт, скомпільований для однієї системи, не запуститься на іншій без повної переробки архітектури контракту;
- в Bitcoin помилка в скрипті просто робить транзакцію недійсною. В Ethereum помилка в контракті може спалити весь газ, але транзакція все одно запишеться в блокчейн. Ці підходи до обробки винятків несумісні на рівні ядра.

Процеси ініціалізації та конфігурації

Процес розгортання блокчейн-платформи починається з налаштування параметрів вузла та визначення початкового стану реєстру. У публічних мережах ці параметри жорстко закодовані в клієнтському ПЗ, тоді як при створенні приватних або тестових мереж розробник має повний контроль над конфігурацією.

Конфігураційні файли та Genesis-блоки

Процес ініціалізації радикально відрізняється для транзакційних систем та обчислювальних платформ.

У Bitcoin-подібних систем, Bitcoin, Litecoin, Dash, основним інструментом конфігурації є файл bitcoin.conf або dash.conf. У цьому файлі вказуються мережеві порти, дозволені IP-адреси для RPC з'єднань, ліміти пам'яті для бази даних та параметри індексації транзакцій.

У Bitcoin генезис-блок вшитий у вихідний код. Він містить відому фразу про канцлера та газетний заголовок від 2009 року. При розгортанні форку, як от

Litecoin, розробники змінюють лише хеш-суму цього блоку та позначку часу, не змінюючи структуру самого процесу ініціалізації.

Ethereum же вимагає значно складнішого підходу через наявність віртуальної машини. Ініціалізація приватної мережі починається з файлу genesis.json.

- config визначає версію протоколу та активні хардфорки, наприклад, London, Shanghai.
- difficulty – початкова складність майнінгу, у приватних мережах зазвичай ставиться низькою для швидкості.
- gasLimit – максимальна кількість обчислювальної роботи, яку може вмістити один блок.
- Alloc – секція попереднього розподілу коштів, де вказуються адреси гаманців та їхній початковий баланс.

Ідентифікація мережі та Node Discovery

Щоб вузли могли спілкуватися між собою, вони повинні ідентифікувати мережу, до якої належать, та знайти сусідів.

В Ethereum параметр chainId є критичним для безпеки. Він використовується для підпису транзакцій, щоб запобігти атак повтору, якщо транзакція була відправлена в тестову мережу, вона не зможе бути виконана в основній. В Bitcoin/Litecoin мережі розділяються на mainnet, testnet та regtest через різні магічні байти у заголовках пакетів даних, що пересилаються по P2P.

Методи виявлення вузлів:

- DNS Seeds – вузли звертаються до заздалегідь визначених DNS-серверів, які повертають списки активних IP-адрес інших вузлів.
- P2P Peer Exchange – після підключення до першого вузла, новий учасник запитує в нього список адрес інших відомих йому сусідів.

- Static Nodes – у приватних мережах, особливо в Dash для мастернод або в NEO для консенсусних вузлів, адреси вузлів часто прописуються вручну в конфігурації для забезпечення стабільного зв'язку.

Механізми емісії та початковий стан

Емісія – це процес створення нових монет. Вона може бути закладена в алгоритм або визначена при ініціалізації.

У Bitcoin, Litecoin, Dash алгоритмічна емісія. Кількість монет не прописана в одному місці. Вона є результатом виконання функції винагороди за блок. У коді прописано: «За кожний блок давати X монет, кожні 4 роки зменшувати цю суму вдвічі». Початкова кількість монет у генезис-блоці Bitcoin дорівнює 50 BTC, але вони технічно не можуть бути витрачені.

У Ethereum, NEO pre-allocation. В Ethereum значна частина монет була створена в генезис-блоці для учасників краудсейлу. В NEO вся емісія , 100 млн токенів, була створена одномоментно при ініціалізації мережі. Половина була розподілена між інвесторами, інша половина – заблокована для розвитку екосистеми. Це принципово відрізняє NEO від Bitcoin, де монети «народжуються» поступово.

Технічний аналіз процесів ініціалізації підтверджує неможливість взаємозаміни модулів:

- модуль ініціалізації Ethereum шукає JSON-структуру з параметрами EVM та лімітом газу. Модуль Bitcoin шукає зашиті в код константи та магічні числа. Спроба ініціалізувати вузол Bitcoin за допомогою genesis.json призведе до критичної помилки синтаксису;
- системи мають різні формати рукостискання. Навіть якщо вузол Bitcoin знайде IP-адресу вузла Ethereum, вони не зможуть встановити з'єднання, оскільки протокол ідентифікації мережі очікує різні структури даних;

- модуль, що відповідає за перевірку винагороди за блок у Dash, де 10% йде в бюджет проекту, а решта ділиться між майнерами та мастернодами, не зможе функціонувати в Litecoin, де вся винагорода належить майнеру.

Аналіз системи керування ресурсами та комісіями

Механізм керування ресурсами визначає, як система розподіляє обмежену пропускну здатність блокчейну між користувачами. Оскільки кожен вузол мережі повинен обробити та зберегти кожну транзакцію, безкоштовне використання мережі призвело б до швидкого перевантаження (DoS-атаки). Кожна з аналізованих систем використовує свій підхід до оцінки "вартості" транзакції.

Ринок комісій за розмір (Bitcoin, Litecoin, Dash)

У класичних блокчейнах основним обмеженим ресурсом є місце в блоці. Оскільки ці системи не виконують складних обчислень, навантаження на процесор вузла є мінімальним, а основне навантаження припадає на пропускну здатність каналів зв'язку та дисковий простір. Комісія розраховується за формулою сатоші за байт. Користувач платить не за суму переказу, а за фізичний обсяг даних, який займає його транзакція в блоці.

Транзакція з багатьма входами, наприклад, збір дрібних сум з десяти різних адрес, буде значно дорожчою, ніж транзакція на велику суму з одного входу. Майнери обирають транзакції з найвищою ціною за байт. Якщо мережа перевантажена, виникає черга і користувачі змушені піднімати ставки, щоб їхня операція потрапила в наступний блок.

Крім стандартних комісій, у Dash існують фіксовані або підвищені комісії за використання функцій InstantSend, де плата йде за швидке підтвердження через мастерноди.

Система Gas

Ethereum, як обчислювальна платформа, не може обмежуватись лише розміром транзакції. Програма, смарт-контракт, може займати лише 100 байт, але виконувати нескінченний цикл, який паралізує роботу всієї мережі.

Поняття Gas – це абстрактна одиниця вимірювання обчислювальної роботи. Кожна операція в EVM, додавання, запис у пам'ять, перевірка підпису, має свою фіксовану вартість у Gas. Користувач встановлює Gas Price . Підсумкова вартість транзакції – це Gas Used × Gas Price.

Якщо в контракті є помилка або нескінченний цикл, виконання зупиниться, як тільки вичерпається ліміт газу, вказаний користувачем. Це робить атаки на обчислювальну потужність мережі надзвичайно дорогими для зловмисника.

Сучасний Ethereum використовує механізм базової плати, яка спалюється, та чайових, які йдуть валідаторам, – це робить комісії більш передбачуваними.

Двохтокенова модель та системний GAS

NEO використовує унікальний підхід, розділяючи права управління мережею та оплату її послуг на два окремі активи:

- NEO, Governance Token, – це неподільний токен, що дає право голосу за консенсусні вузли та право на отримання дивідендів у вигляді GAS.
- GAS, Utility Token, – це подільний токен, який використовується для оплати транзакційних комісій та розгортання смарт-контрактів.

У системі NEO деякі типи транзакцій можуть бути безкоштовними в межах певних лімітів, але складні операції, такі як розгортання нового смарт-контракту, вимагають значної кількості GAS. Це стимулює розробників писати оптимізований код та запобігає засміченню реєстру непотрібними контрактами.

На відміну від Ethereum, де газ спалюється або йде валідаторам, у NEO GAS генерується кожним блоком і розподіляється між власниками токенів NEO, що створює іншу економічну динаміку розгортання вузлів.

Аналіз механізмів комісій остаточно підтверджує неможливість взаємозаміни модулів.

- Модуль Bitcoin вміє рахувати лише байти. Якщо його вставити в Ethereum, він дозволить виконувати надважкі обчислення за безцінь, що призведе до миттєвої зупинки мережі через перевантаження процесорів.
- В Ethereum комісія знімається з того самого рахунку, що й основний актив. В NEO транзакція може вимагати наявності іншого типу активу. Модуль перевірки балансу Bitcoin просто не зрозуміє концепцію двох різних балансів для однієї операції.
- Модуль керування ресурсами Ethereum жорстко прив'язаний до специфікацій EVM. Оскільки в Bitcoin немає віртуальної машини, поняття вартості операції додавання для нього не має сенсу.
- Модуль Dash, що працює з мастернодами для InstantSend, вимагає специфічного розподілу комісій, частина йде майнеру, частина — мастерноді. У Bitcoin та Litecoin такого розподілу не існує, що робить модуль Dash непрацездатним у цих мережах.

Таким чином, кожна система має унікальний скажімо так імунітет до перевантажень, який неможливо відокремити від її архітектурного ядра.

Висновки: в ході виконання лабораторної роботи було досліджено технічні особливості розгортання блокчейн-платформ, проведено порівняльний аналіз транзакційних моделей і обґрунтувано архітектурну несумісність їхніх програмних модулів.